# The stable manifold method for approximate solution of the Hamilton-Jacobi equation

— Guide for Matlab program set

N. Sakamoto


| | |
|---|---|
| 1st version | February 23, 2011 |
| 2nd version | March 29, 2011 |
| 3rd version | Octorber 10, 2013 |
| 4th version | April 26, 2017 |
| 5th version | December ?, 2023 |


## Contents


**1 The Hamilton-Jacobi equation and stable manifold** **1**

**2 The stable manifold algorithm** **3**

**3 Program structure** **6**

**4 Program description and important variables** **12**

4.1 `P1_data_setting.m` . . . . . 12

4.2 `P2_hamiltonian_check.m` . . . . . 14

4.3 `P3_decide_xi.m` . . . . . 15

4.4 `P4_iterative_calcu.m` . . . . . 15

4.5 `P5_solution_check.m` . . . . . 16


# 1 The Hamilton-Jacobi equation and stable manifold

$$\text{(HJ)} \qquad H(x,p) = p^T f(x) - \frac{1}{4} p^T \bar{R}(x) p + q(x,p) = 0,$$

where $p_1 = \partial V/\partial x_1, \cdots, p_n = \partial V/\partial x_n$ with $V(x)$ an unknown function, $f : M \to \mathbb{R}^n$, $\bar{R} : M \to \mathbb{R}^{n\times n}$, $q : M \times \mathbb{R}^n \to \mathbb{R}$ are all $C^\infty$, and $\bar{R}(x)$ is a symmetric matrix for all $x \in M$. We also assume that $f$ and $q$ satisfy $f(0) = 0$, $q(x,p) \geqslant 0$, $q(0,0) = 0$ and $q(x,p) = o(|x| + |p|^2)$. In what follows, we write $f(x) = Ax + o(|x|)$ where $A = \frac{\partial f}{\partial x}(0)$ is an $n \times n$ real matrix. A typical control problem in which (HJ) appears is the optimal regulation problem that this toolbox aims at.

***Optimal regulation problem:***

$$\begin{aligned} \dot{x} &= f(x) + g(x)u \\ J &= \int_0^\infty x^T Q x + u^T R u \, dt, \end{aligned} \tag{1}$$

where $R > 0$ and $Q \geqslant 0$. We will derive (HJ) using dynamic programming (DP) and an algebraic Riccati equation that is a linear approximation of (HJ). The pre-Hamiltonian for DP is

$$H_D(x,p,u) = p^T(f(x) + g(x)u) + x^TQx + u^TRu$$

and the minimizing vector $u$ for $H_D$ is

$$\bar{u}(x,p) = -\frac{1}{2}R^{-1}g(x)^Tp.$$

Substituting this to $H_D$, we get a form of (HJ)

$$H(x,p) = H_D(x,p,\bar{u}(x,p)) = p^Tf(x) - \frac{1}{4}p^Tg(x)R^{-1}g(x)^Tp + x^TQx = 0.$$

Note that $\bar{R}(x) = g(x)R^{-1}g(x)^T$ and $q(x,p) = x^TQx$. If one obtains the stabilizing solution $V(x)$ to this Hamilton-Jacobi equation, the optimal control is given by

$$u_{\text{opt}} = \bar{u}(x, \partial V/\partial x) = -\frac{1}{2}R^{-1}g(x)^T\frac{\partial V}{\partial x}(x)^T.$$

To get a linear approximation of this HJ, we collect quadratic terms:

$$x^T\left(PA - \frac{1}{4}PBR^{-1}B^TP + Q\right)x = 0,$$

where we have set $V(x) = \frac{1}{2}x^TPx + o(|x|^2)$, $p = (\partial V/\partial x)(0)^T = x^TP + o(|x|)$ and $B = g(0)$. This, using $x^TPAx = \frac{1}{2}(x^TPAx + x^TA^TPx)$, is written as

$$\frac{1}{2}x^T(PA + A^TP - PB(2R)^{-1}B^TP + 2Q)x = 0,$$

thus obtaining an algebraic Riccati equation

$$PA + A^TP - PB(2R)^{-1}B^TP + 2Q = 0.^1$$

**Definition 1.** *A solution $V(x)$ of* (HJ) *is said to be the stabilizing solution if $p(0) = 0$ and $0$ is an asymptotically stable equilibrium of the vector field $\frac{\partial H}{\partial p}(x,p(x)) = f(x) - \frac{1}{2}\bar{R}(x)p(x)$, where $p(x) = (\partial V/\partial x)^T(x)$.*

Suppose $V(x)$ is a solution of (HJ). Then, the set

$$\Lambda_V = \{(x,p)\,|\,p = \partial V/\partial x(x)\}$$

is invariant under the flow of the associated Hamiltonian system derived from (HJ):

$$\begin{cases} \dot{x} = f(x) - \dfrac{1}{2}\bar{R}(x)p \\ \dot{p} = -\dfrac{\partial f}{\partial x}(x)^Tp + \dfrac{1}{4}\dfrac{\partial (p^T\bar{R}(x)p)^T}{\partial x} - \dfrac{\partial q}{\partial x}^T(x,p). \end{cases} \tag{2}$$

Conversely, if an $n$-dimensional manifold $\Lambda$ in the $(x,p)$-space is invariant under (2) and, at a point $(x_0,p_0)$, the projection $\pi$ of $\Lambda$ to the $x$-space is surjective, then, $\Lambda$ possesses the Lagrangian submanifold property and there exists a solution $V(x)$ of (HJ) around $x_0$. Let

[1] The coefficient 1/2 and 2 in front of $R$ and $Q$ are to be consistent with the program set (see `P1_data_setting.m`)

$(x_0, p_0) = (0, 0)$. If, in addition to the above conditions, the Hamiltonian flow of (2) on $\Lambda$ is convergent to the origin, then, $\Lambda$ is a stable manifold of (2) and there exists a function $V(x)$ defined on a neighborhood $U$ of the origin such that

$$\Lambda \cap \pi^{-1}(U) = \{(x, p) \,|\, p = \partial V/\partial x(x),\ x \in U\}.$$

A sufficient condition for the local existence of the stable manifold, or, equivalently, the local stabilizing solution for (HJ) is obtained in [5]. It is a natural condition based on a linearization argument. Let us consider the Riccati equation

$$\text{(RIC)} \qquad PA + A^T P - \frac{1}{2} P\bar{R}(0)P + 2Q = 0,^2$$

which is the linearization of (HJ). A symmetric matrix $P$ is said to be the stabilizing solution of (RIC) if it is a solution of (RIC) and $A - \frac{1}{2}\bar{R}(0)P$ is stable.

**Theorem 2** (van der Schaft, 1991)**.** *If (RIC) has the stabilizing solution $P$, there exists, locally around the origin, the stabilizing solution $V(x)$ to (HJ) with $(\partial^2 V/\partial x^2)(0) = P$.*

The theorem also says that if the stabilizing solution to (RIC) exists, an $n$-dimensional stable manifold, from which the stabilizing solution to (HJ) can be derived, locally exists around the origin (See Figure 1). Throughout this note, we assume that the stabilizing solution to (RIC), denoted by $\Gamma$, exists. As is described earlier, an $n$-dimensional stable manifold $\{(x, p) \,|\, p = p(x)\}$ exists for (2) around the origin. The function $p(x)$ is used in feedback laws and our problem now is how to approximate $p(x)$.

Let

$$T = \begin{pmatrix} I & S \\ \Gamma & \Gamma S + I \end{pmatrix}, \tag{3}$$

where $S$ is the solution of the Lyapunov equation $(A - \frac{1}{2}\bar{R}(0)\Gamma)S + S(A - \frac{1}{2}\bar{R}(0)\Gamma)^T = \bar{R}(0)$. Using the linear coordinate transformation

$$\begin{pmatrix} x' \\ p' \end{pmatrix} = T^{-1} \begin{pmatrix} x \\ p \end{pmatrix},$$

the Hamiltonian system (2) is written

$$\begin{pmatrix} \dot{x}' \\ \dot{p}' \end{pmatrix} = \begin{pmatrix} A - \frac{1}{2}\bar{R}(0)\Gamma & 0 \\ 0 & -(A - \frac{1}{2}\bar{R}(0)\Gamma)^T \end{pmatrix} \begin{pmatrix} x' \\ p' \end{pmatrix} + \text{higher order terms.} \tag{4}$$

# 2 The stable manifold algorithm

This section describes the stable manifold approach for solving (HJ) with a successive approximation method[4]. Let us consider the following $2n$-dimensional system.

$$\begin{cases} \dot{x} = Fx + n_s(t, x, y) \\ \dot{y} = -F^T y + n_u(t, x, y). \end{cases} \tag{5}$$

We assume that $F$ is an asymptotically stable $n \times n$ real matrix and $n_s$, $n_u$ are higher order terms with sufficient smoothness.

[2]The coefficients here are not standard ones with the same reason above.

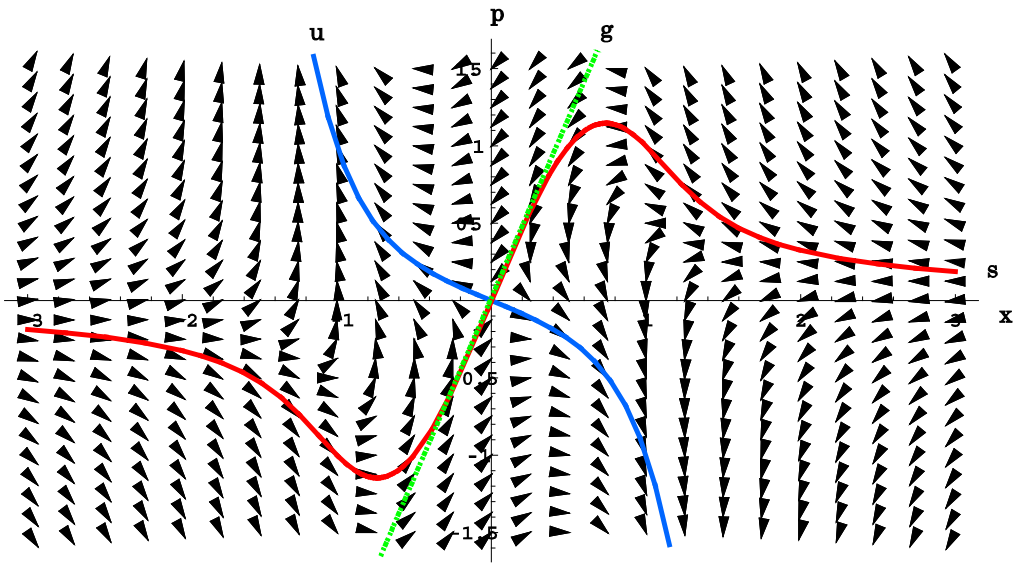



Figure 1: Stable and unstable manifolds (red and blue, respectively) of a 2-dimensional Hamiltonian flow. The green line tangent to the stable manifold at the origin corresponds to the stabilizing solution to an associated Riccati equation.

We define the sequences $\{x_k(t,\xi)\}$ and $\{y_k(t,\xi)\}$ by

$$\begin{cases} x_{k+1} = e^{Ft}\xi + \displaystyle\int_0^t e^{F(t-s)} n_s(s, x_k(s), y_k(s))\, ds \\ y_{k+1} = -\displaystyle\int_t^\infty e^{-F^T(t-s)} n_u(s, x_k(s), y_k(s))\, ds \end{cases} \tag{6a}$$

for $k = 0, 1, 2, \ldots$, and

$$\begin{cases} x_0 = e^{Ft}\xi \\ y_0 = 0 \end{cases} \tag{6b}$$

with arbitrary $\xi \in \mathbb{R}^n$.

**Theorem 3** (Sakamoto and van der Schaft, 2008)**.** *The sequences $x_k(t,\xi)$ and $y_k(t,\xi)$ are convergent to zero for sufficiently small $|\xi|$, that is, $x_k(t,\xi)$, $y_k(t,\xi) \to 0$ as $t \to \infty$ for all $k = 0, 1, 2, \ldots$ Furthermore, $x_k(t,\xi)$ and $y_k(t,\xi)$ are uniformly convergent to a solution of (5) on $[0,\infty)$ as $k \to \infty$ for sufficiently small $|\xi|$. Let $x(t,\xi)$ and $y(t,\xi)$ be the limits of $x_k(t,\xi)$ and $y_k(t,\xi)$, respectively. Then, $x(t,\xi)$, $y(t,\xi)$ are the solution on the stable manifold of (5), that is, $x(t,\xi)$, $y(t,\xi) \to 0$ as $t \to \infty$.*

For each $k$, the functions in (6) are calculated for the system (4) to obtain $x_k'(t,\xi)$ and $p_k'(t,\xi)$. Functions $x_k(t,\xi)$ and $p_k(t,\xi)$ given by

$$\begin{pmatrix} x_k(t,\xi) \\ p_k(t,\xi) \end{pmatrix} = T \begin{pmatrix} x_k'(t,\xi) \\ p_k'(t,\xi) \end{pmatrix}$$

form parametrizations of the approximate stable manifold of (4). More precisely, the stable manifold $\Lambda$ of (4) is approximated by

$$\Lambda_k = \{(x_k(t,\xi), p_k(t,\xi)) \,|\, |\xi| \text{ small},\ t \geqslant 0\}$$

with $k$ sufficiently large. Furthermore, if one obtains a function $\pi_k(x)$ such that

$$\Lambda_k = \{(x, \pi_k(x)) \,|\, x \text{ in a neighborhood of } 0\},$$

the approximate optimal control is given by

$$u_k = -\frac{1}{2} R^{-1} g(x)^T \pi_k(x).$$

In this subsection, we describe how to compute (6) and obtain the approximate stable manifold in the form that feedback laws $u_k(x)$ can be easily derived.

To explain the details of the computations, we use a simple 1-dimensional optimal control problem.

$$\dot{x} = x - x^3 + u$$
$$J = \int_0^\infty \frac{q}{2} x^2 + \frac{r}{2} u^2 \, dt.$$

The Hamilton-Jacobi equation for this problem is

$$H = p(x - x^3) - \frac{1}{2r} p^2 + \frac{q}{2} x^2 = 0 \tag{7}$$

and Hamilton's canonical equations are

$$\begin{cases} \dot{x} = x - x^3 - \dfrac{1}{r} p \\ \dot{p} = -(1 - 3x^2) p - qx. \end{cases}$$

Actually, the Hamilton-Jacobi equation (7) can be solved exactly and therefore, it is easy to see how computations proceed, approaching an exact solution. The following are key parameters and tasks necessary to perform the computations.

*Radius of convergence:* One of the most important tasks in the computation of the sequences (6) is to determine the region for $\xi$ such that the convergence of the functions is guaranteed. Fig. 2 shows that when $|\xi|$ is too large, the sequences (6) are not convergent.

*Domain enhancement by negative time:* Since the functions in (6) go to the origin for $t \to \infty$, the resultant stable manifold is in the radius of convergence, which is too small for practical use (see, Fig 3). This problem is circumvented by using negative $t$ in (6). This idea is based on the invariance of the stable manifold. Suppose we have the limit functions $x(t, \xi)$, $p(t, \xi)$. The invariance property suggests that, for $t < 0$, $x(t, \xi)$ and $p(t, \xi)$ are still on the stable manifold as long as it exists. For finite approximation $x_k(t, \xi)$, $p_k(t, \xi)$, this tendency is more present as higher approximation is employed (see, Fig. 4). For actual computation, we take a large $k$ and see how long the functions are extended in the negative direction by checking the value of Hamiltonian (see, the next item).

*Hamiltonian check:* Hamiltonian is the left side of (HJ) and its value can be used as a measure for the accuracy of approximation (see, Fig. 5). That the value of Hamiltonian is not close to zero means that the solution curve is not on the stable manifold and such data cannot be used in the feedback construction.

*Surface construction:* In this approach, each solution curve that is on the stable manifold is computed. By collecting these lines, the function $p(x)$ is approximated (See Figures 6 and 7). There are several ways to do this. One is to use $n$-dimensional interpolation, another one is to use $n$-variable polynomial fitting. The latter is employed in this paper due to its simplicity for computation and implementation to the system under control. The program is based on Polybase by Giampiero Campa at
`http://www.mathworks.com/matlabcentral/fileexchange/15`.

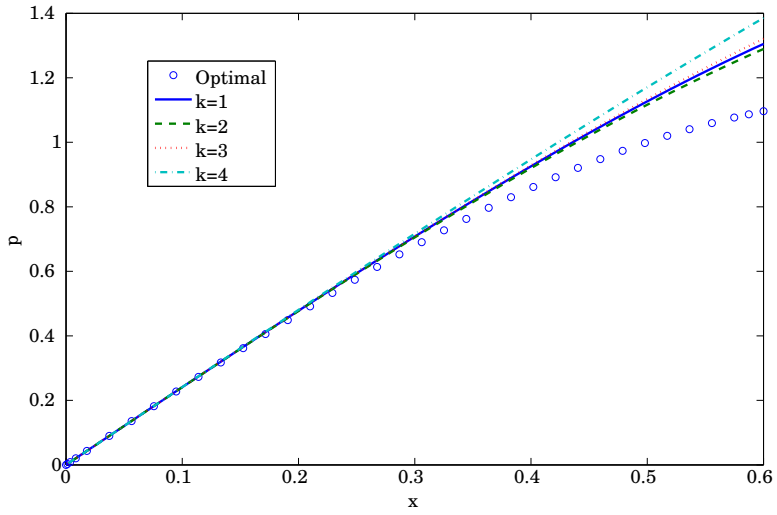



Figure 2: Plots of $x_k(t, \xi)$ and $p_k(t, \xi)$ with $t \geqslant 0$ for $\xi$ outside of the radius of convergence and the optimal $p(x)$

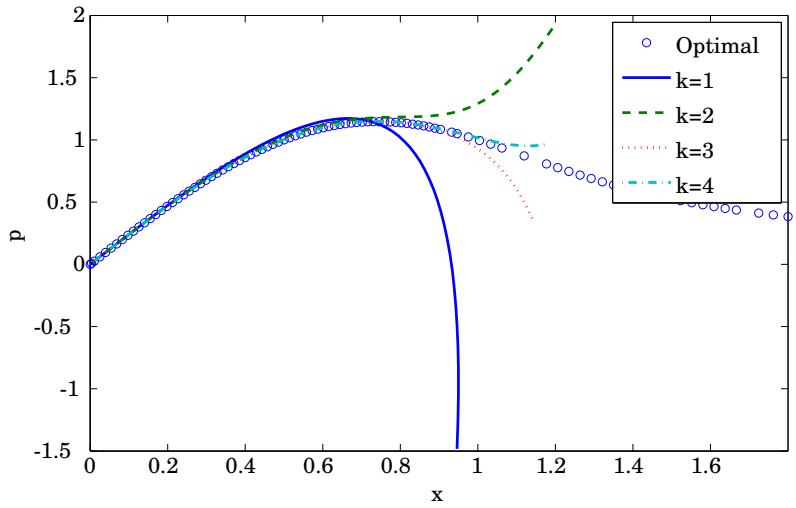



Figure 3: Plots of $x_k(t, \xi)$ and $p_k(t, \xi)$ for $t \geqslant 0$ and the optimal $p(x)$

# 3 Program structure

The programset is named as `NonlinearControltoolbox` and in a folder with the following trees.

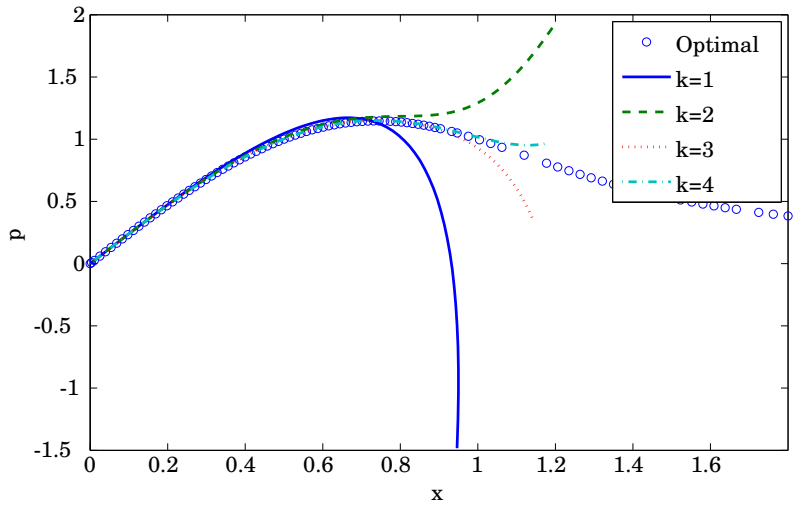



Figure 4: Plots of $x_k(t, \xi)$ and $p_k(t, \xi)$ for $t \geqslant -0.8$ and the optimal $p(x)$

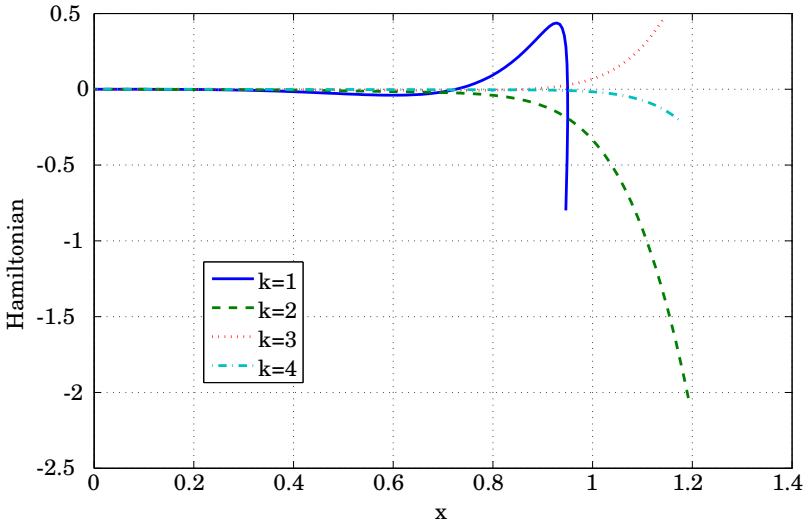
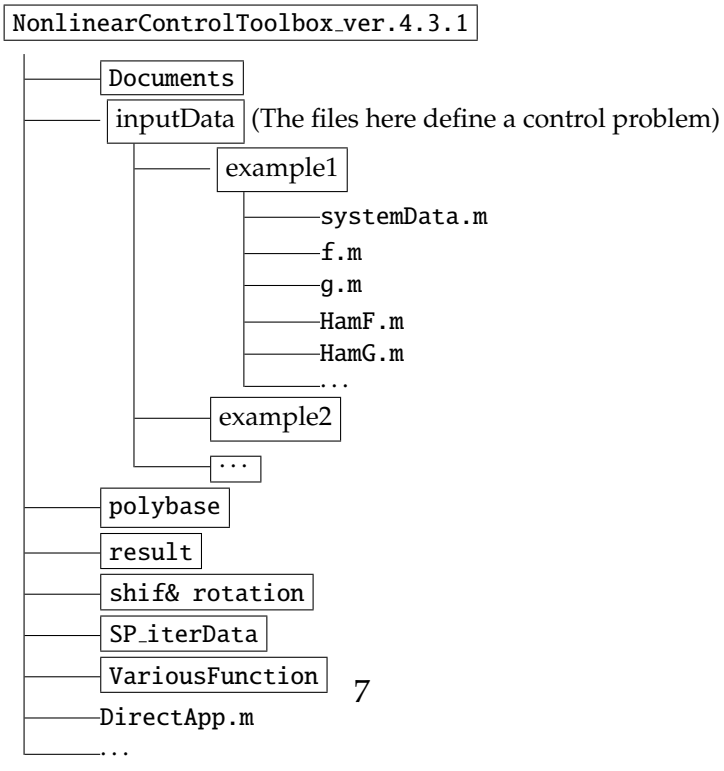



Figure 5: Values of Hamiltonian for each $k$



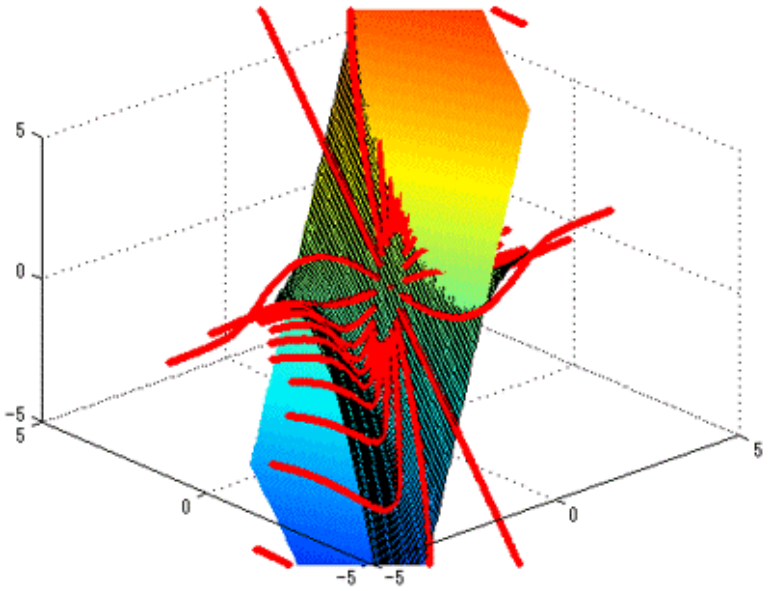



Figure 6: Iteratively computed lines $(x_k(t,\xi), p_k(t,\xi))$ and created surface (hashed surface). The flat plain is the Riccati solution.

The main programs are in `NonlinearControltoolbox` which should not be modified. The files that define each problem are in `inputData`. `VariousFunction` folder contains several important m-files such as `ordblk.m, reig.m` to compute Jordan canonical forms.

The program that computes approximate solutions of the Hamilton-Jacobi equation is a program group with the following structure.

- `DirectApp.m`
  - `P1_data_setting.m`
  - `P2_hamiltonian_check.m`
    - `SP_appNth.m`
  - `P3_decide_xi.m`
    - `Decide_xi.m` (optional)
  - `P4_iterative_calcu.m`
    - `termination_pos.m`
    - `termination_neg.m`
  - `P5_solution_check.m`
    - `SP_shooting_plot.m`
    - `SP_data_conv.m`
    - `SP_ham_calcu.m`
    - `SP_graphplot1.m`
    - `SP_data_cut.m`
    - `SP_griddata.m`
    - `SP_multiPoly.m`
    - `SP_response_calcu.m`
    - `(SP_RateLimit_Linear_Response.m)`
    - `SP_norm_calcu.m`

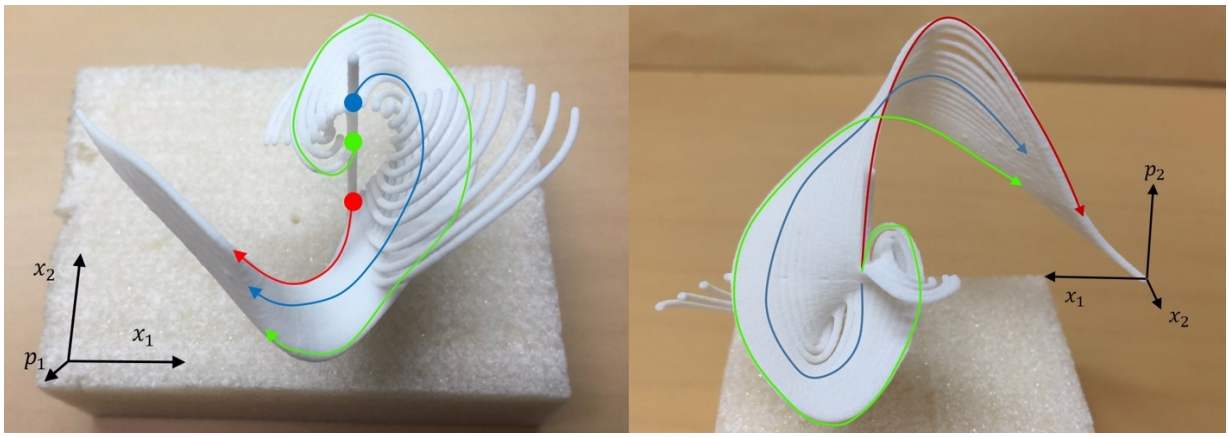



Figure 7: A 3D figure of stable manifold computed for pendulum swingup problem[2]. Left: $x_1$-$x_2$-$p_1$ space, Right: $x_1$-$x_2$-$p_2$ space.

The main program `DirectApp.m` calls 5 principal programs `P1_...m`, ..., `P5_...m`. These 5 principal programs consist of several subprograms `SP_...m`. In these programs, there are frequently used utility functions (`f_...m`) for certain purposes and ode integrands (`i_...m`). Some of important functions are listed below.

utility functions

| | |
|---|---|
| `f_hamcalcu.m` | computes values of Hamiltonian: $p^T f(x) - \frac{1}{4} p^T \bar{R}(x) p + q(x,p)$<br>It is used to see if the computed data is actually on the stable manifold<br>(or, satisfies the Hamilton-Jacobi equation) |
| `f_costopt.m` | computes the cost value of obtained feedback control |

ode integrands

| | |
|---|---|
| `i_directforx.m` | 1st component of right-side in (2)<br>used in positive computation |
| `i_directforp.m` | 2nd component of right-side in (2)<br>used in positive computation |
| `i_canonical_eq.m` | Hamiltonian system (2), $2n$-dimensional vector field<br>used in negative computation |
| `i_state_eq_for_control.m` | closed-loop system<br>used to evaluate the performance of the controller |

The stable manifold method proposed in [4] consists of 5 steps[3], which are described in some detail below.

(1) **Data settings and preliminary calculations** (`P1_data_setting.m` and files in `InputData` folder):

1.1) **Data input**: In the first process, system parameters and cost functions should be assigned. Suppose that the system under control and the cost function are

$$\dot{x} = f(x) + g(x)u \tag{8}$$
$$J = \int_0^\infty x^T Q x + u^T R u \, dt.$$

Then, `systemData.m` requires $A$, $B$, $Q$ and $R$, where

$$f(x) = Ax + O(|x|^2),\ A = \frac{\partial f}{\partial x}(0), \quad B = g(0).$$

In `HamF.m` and `HamG.m`, nonlinear terms of the Hamiltonian system (2) are given. Namely,

$$\texttt{HamF.m} = f(x) - Ax - \frac{1}{2}\bar{R}(x)p + \frac{1}{2}\bar{R}(0)p$$
$$\texttt{HamG.m} = -\frac{\partial f}{\partial x}(x)^T p + \frac{1}{4}\frac{\partial (p^T \bar{R}(x)p)}{\partial x}^T - \frac{\partial q}{\partial x}^T (x,p) + \frac{\partial f}{\partial x}(0)^T p + \frac{1}{2}\frac{\partial^2 q}{\partial x^2}(0,0)x$$

These must be carefully computed and manually coded in `systemData.m`. Also,

$$\texttt{f.m} = f(x) \quad \text{and} \quad \texttt{g.m} = g(x)$$

These functions are used in the Hamiltoinan calculation in `SP_ham_calcu.m` for data verification and must be compatible with `HamF.m` and `HamG.m`.

1.2) **Preliminary calculations**: In this process, the approximating Riccati equation for (HJ) is solved as $P$ and the diagonalizing matrix $T$ in (3) is obtained. Using the Riccati solution and a solution of a Lyapunov equation, the block diagonalized Hamiltonian system (4) is obtained. Also, eigenvalues and real Jordan canonical form of of $F := A - \frac{1}{2}\bar{R}(0)P$ is computed with `ordblk.m` in the folder `VariousFunctions`. The output `Trs` of `ordblk.m` contains eigenvectors of $F$.

(2) **Determination of the radius of convergence** (`P2_hamiltonian_check.m`): One of the most important parameters in the stable manifold method (iteration) is the radius $|\xi|$ in (6b) and it is given as `radi` in the toolbox. By examining the proof of Theorem 3, one notices that stable eigenvalues of $F$ dictate the convergence.

The program `SP_appNth.m` is run for each eigenvector of $F$ with different `radi`. When `radi` is moved from small to large, with enough iterations, values of Hamiltonian $H(x(0;\xi), p(0;\xi))$ suddenly go up from a certain value. The boundary of this change can be considered as the radius of convergence for the iteration. The values of this boundary are different in each eigenvector. But, it is generally true (but exceptions do exist due to combinations with nonlinearity) that the boundary of the eigenvalue that is closest to the imaginary axis is smallest and this can be used as the radius of convergence for the whole iterations. In the computation with dialogue mode (when `xiRFlag==0`), a figure like Fig. 5 is shown asking if the value of `radi` is appropriate.

(3) **Creation of the set of initial points $\xi$'s** (`P3_decide_xi.m`): The set of initial points $\xi$ in (6b) should be formed in such a way that corresponding trajectories $x_k(t;\xi)$ uniformly distribute in $x$-space and the area formed by $x_k(t;\xi)$ covers the area in which the control problem should be solved. In `P3_decide_xi.m`, $\xi$'s are distributed on the sphere with the radius `radi` whose principal axes correspond to the eigenvectors of $F$. One can take uniformly distributed mesh on the sphere by changing parameters in `systemData.m`. But, this program handles only dimension upto 5. One can select an option `iniFlag` to independently provide the set of initial points in `Decide_xi.m`.

(4) **Computation of solution curves of the Hamiltonian systems** (`P4_iterative_calcu.m`): This corresponds to calculate solutions on stable manifold of (2). This program is carried

out for each $\xi$ in the set of initial points made by `P3_decide_xi.m`. First of all, the stable manifold iteration is applied to

$$\begin{cases} \dot{x} = (A - \frac{1}{2}\bar{R}(0)P)x + n_s(x, y) \\ \dot{y} = -(A - \frac{1}{2}\bar{R}(0)P)^T y + n_u(x, y). \end{cases}$$

In `P4_iterative_calcu.m`, the nonlinear functions $n_s$, $n_u$ are defined in `i_highf.m` and `i_highg.m`, respectively, using $T$, $T^{-1}$, `HamF.m` and `HamG.m`.

The sequences $x_n(t)$, $y_n(t)$ in (6), in this case, satisfy

$$\begin{cases} \dot{x} = (A - \frac{1}{2}\bar{R}(0)P)x + n_s(x_{n-1}(t), y_{n-1}(t)), \quad x(0) = \xi \\ \dot{y} = -(A - \frac{1}{2}\bar{R}(0)P)^T y + n_u(x_{n-1}(t), y_{n-1}(t)), \quad y(\infty) = 0. \end{cases} \tag{9}$$

Note that these equations are separated non-homogeneous linear equations. They are defined in `i_directforx.m` and `i_directforp.m`. In actual computations, $\infty$ is replaced by a sufficiently large value $T =$ `stoptime` and the second equation is solved as an initial value problem with reversed time for $[$`stoptime`$, 0]$ with $y($`stoptime`$) = 0$. The computed functions $x_n(t)$, $y_n(t)$ are substituted in the same equations in the next step. In Matlab, $x_n(t)$, $y_n(t)$ are obtained over sampled time values. So, in `i_directforx.m` and `i_directforp.m`, the continuous functions are constructed from this data by the command `interp1q`.

For sufficiently small $|\xi|$, $x_n(t;\xi)$, $y_n(t;\xi)$ converge to the solution on the stable manifold (in the coordinates in which the linear part is block diagonalized). Also $x_n(t;\xi)$, $y_n(t;\xi) \to 0$ as $t \to \infty$. Then, the stable manifold is approximately parameterized as

$$\{(x_n(t;\xi), y_n(t;\xi) \,|\, t \geqslant 0,\ |\xi| : \text{small}\} \tag{10}$$

Since $\xi$ is small, this manifold is generally small around the origin. To enhance the computed stable manifold, the solutions for negative time domain should be used. Specifically, (4) is solved for time interval $[t_{\text{neg}}, 0]$ with $t_{\text{neg}} < 0$ as an initial value problem $x(0) = x_{k_{\max}}(0)$, $p(0) = p_{k_{\max}}(0)$, where $x_{k_{\max}}(t)$, $p_{k_{\max}}(t)$ are the solutions from the last iteration in the positive time computation. This computation is done in the line
`ode45(@i_canonical_eq, [Time_s Tneg], Tneg_ini_xpT, options)`
For this negative directional computation, no iteration is necessary. As long as the positive directional computation is obtained with good accuracy, which means that $(x_{k_{\max}}(0), p_{k_{\max}}(0))$ is on the stable manifold, the solution for negative direction never leaves from the manifold due to its invariance property.

(5) **Controller construction and examination** (`P5_solution_check.m`): First, note that the controllers of optimal control problems are given by

$$u = -\frac{1}{2}R^{-1}g(x)^T p(x),$$

where, $R$ is a positive definite matrix in the cost function, $g(x)$ is the system input matrix and $p(x) = (\partial V/\partial x)^T$ with the stabilizing solution $V(x)$ of (HJ). Thus, controller construction is equivalent to the construction of function $p(x)$. This is done by obtaining the functional relation between $x$ and $p$ on the computed stable manifold

$$\{(x_n(t;\xi), p_n(t;\xi) \,|\, t \in [t_{\text{neg}}, \texttt{stoptime}],\ |\xi| : \text{small}\}. \tag{11}$$

Note that the stable manifold (11) is a transformed one from (10) by the diagonalizing matrix $T = \begin{pmatrix} I & S \\ \Gamma & \Gamma S + I \end{pmatrix}$ in (3).

5.1) **Checking out the data distribution in $x$-space**: It is desirable that the stable manifold (11) covers the area in $x$-space in which the control problem at hand should be solved. Also, the data constituting (11) should distribute uniformly. For now, this checking is done manually.

5.2) (`SP_ham_calcu.m`) Before the construction of the controller, or, functional relation $p(x)$, it is possible to see how well $p(x)$ approximates the solution of (HJ). This is done simply by substituting the data of (11) in the left-side of (HJ) and draw $x-H$ graphs (see Fig. 5). The area in which $H = 0$ can be considered as the valid domain of the approximation. If the valid domain is not big enough, higher iterations and more negative $t_{\text{neg}}$ must be employed.

5.3) (`SP_multiPoly.m` or `SP_griddata.m`) The construction of $p(x)$ is possible in two ways. The recommended method is to use polynomial fitting using `SP_multiPoly.m`. This is a modification of Polybase by Giampiero Campa. The modification was made from the following viewpoints. Firstly, $p(0) = 0$ must be satisfied from the property of (HJ) and the constant term should always be set 0. Secondly, it always holds that $p(x) = Px + O(|x|^2)$ meaning that the coefficients of first order terms coincide with the solution of the Riccati equation. Thirdly, if it is possible, by the Taylor expansion method[1], to obtain the coefficients of higher order terms, say, 2nd or 3rd order terms, one wishes to exclude those coefficients from the equations to be solved in Polybase. By doing these, it is possible to reduce the computational load and to obtain more accurate solutions around the origin. This data fitting is basically by the least square method, thus, the function is more accurate in the region with more data. But, we know that in the neighborhood of the origin, where all the solutions accumulate, the function is almost linear. Therefore, data around origin should be removed from the data set, which is done by `SP_data_cut.m` by specifying `cut_time`. The data for $t >$ `cut_time` is removed in the solution $x(t)$, $p(t)$. Finally the gap between the value $p$ in data and the value $p(x)$ by the constructed function is shown asking the necessity to change the order of the polynomial.

5.4) (`SP_response_calcu.m`, `SP_norm_calcu.m`) These programs carry out simulations of the closed loop system for a given initial condition and compute the value of the cost function.

# 4 Program description and important variables

## 4.1 `P1_data_setting.m`

Loading initial data and preliminary computations such as solving a Riccati equation and a Lyapunov equation. Thenk block diagonalization (4) for the Hamiltonian system is performed. Also, the real Jordan canonical form and its transformation matrix are obtained.

#### HamF.m, HamG.m

These files define the higher order terms in (4). The first $n$ elements are in `HamF.m` and the rest of $n$ elements are in `HamG.m`. Note that these higher order temrs are from the Hamiltonian system before the block-diagonalization.

**f.m, g.m**

These files define control system (8). These are not used in the main computation of `P4_interative_calcu.m`, but, the Hamiltonian (the right side of (HJ)) is defined with them in `SP_ham_calcu.m`. Be sure that these define the same problem as the one `HamF.m`, `HamG.m` define.

**SystemData.m**

Data file that describes the HJ equation to be solved.

`satFlag`: Option for saturation function (0: no saturation, 1: saturation exists, 2: rate limiter exists)

`dbFlag`: Option for figure display for debuging (0: no figure, 1: figure displayed)

`meshFlag`: Option for approximation method for $p(x)$ (0: multi-variable polynomial, 1: interpolation)

`iniFlag`: Option for where to provide initial points $\xi$'s ( 0 : in `systemData.m` or in Matlab command window, 1: in `Decide_xi.m`)

`kFlag`: Option for where to provide the iteration number (0: in Matlab command window, 1: in `systemData.m`)

`xiRFlag`: Option for where to provide the radius of convergence (0: in Matlab command window, 1: in `systemData.m`)

`xiTHFlag`: Options for where to provide samples in circumferential direction on the sphere with the radius of convergence (0: in Matlab command window, 1: in `systemData.m`)

`A,B`: Linear parts of the system under control

`Q,R`: Weight matrices in the cost function

`dim`: System dimension

`knum`: The number of iterations(= $k$). $k = 0$ corresponds to linear (Riccati) solution

`radi`: The radius of convergence

`thetaN`($N = 4, 3, ..1$): The circumferential variables in polar coordinate on the initial sphere to define $\xi$'s.

`Tneg_max`: The maximum interval in the negative direction integration

`HAMK_n_max_juddge`: The negative direction integration is terminated when Hamiltonian exceeds this value

`tol`: We interpret that $T = \infty$ is achieved when integrands are small than this value. Used in positive direction infinite interval integrations (6).

`options`: Options for `ode45, ode15s`

`param_cull_pos`: Computed data $x_k, p_k$ is often too crowded and thining out is necessary. This parameter defines the frequency of thining out the data for positive directional integrations. This operation reduces the computation time of $p(x)$.

`param_cull_neg`: Frequency of thining out data for negative directional integrations.

`top_btm`: Value of cut-off function for nonlinearities in negative integrations.

`F`: Diagonalized linear part in (HJ)

`Ft`: Transpose of `F`

`Diagonalize`: Block diagonalization matrix $T$

`Diag_inv`: Inverse of `Diagonalize`; $T^{-1}$

`Ham`: Hamiltonian matrix

`eigF,Trs`: Real Jordan form of $F$ and its transformation matrix. `Trs` contains eigenvectors of $F$ and the one corresponding an eigenvalue closest to the imaginary axis is used to determine `radi`.

*Report in P1:*

| **Items** | **Notes** |
|---|---|
| min & max of the real(`eig(F)`) | if min is close to zero, the convergence of iteration is slow |
| max(real(`eig(F)`)/min(real(`eig(F)`)) | stiffness of $F$ which affects the difficulty of getting uniform data |
| `det(Trs)` | When this is 0 (it happens for unobservable systems), `radi` determination needs to be done carefully |

## 4.2 `P2_hamiltonian_check.m`

By examining the values of Hamiltonian, this program helps find the radius of convergence for the stable manifold algorithm. The program `SP_appNth.m` is run for each eigenvector of $F$ (vector in `Trs`) with different scales (by changing `radi`).

`init_sum_P2`: The set of test initial value $\xi$'s. The eigenvectors of $F$ (vector in `Trs`).

`ini`: Element of `init_sum_P2`. Initial conditions ($\xi$'s) for test iterations to determine `radi`.

`xp_t0_D`: Values of $x$ and $p$ at $t = 0$ for each $\xi$ in `init_sum_P2` in the original space

`HamkN`: The values of Hamiltonian corresponds to`xp_t0_D`

*Report in P2:*

| **Items** | **Notes** |
|---|---|
| min & max of Ham | make sure that max $\leqslant 10^{-7}$ |
| min & max of `stoptimes` | if $F$ is stiff, this gap is large and difficult to get uniform data |
| radius | value of `radi` |
| num of iteration | value of `knum` |

## 4.3 P3_decide_xi.m

This program provides the set of initial values $\xi$'s for the stable manifold algorithm. One can define it in `systemData.m` or directly provide it in `Decide_xi.m` (with flag `iniFlag`=1). The set defined in `systemData.m` is sample points in the sphere with radius `radi` whose axes correspond to eigenvectors of $F$.

`radi`: The radius of convergence

`thetaN?`: Circumferential variables on the sphere with `radi`. Given as a vector form, corresponding samples in each variable.

`init_sum`: The set of $\xi$'s

$$\xi_i = [x_1(0), x_2(0), ..., x_n(0)], n = \text{system dimension}$$
$$\texttt{init_sum} = \left[\xi_1^T, \xi_2^T, ..., \xi_m^T,\right]^T, m = \text{the number of whole samples } \xi\text{'s}$$

*Report in P3:*

| **Items** | **Notes** |
|---|---|
| num of `xi_sum` | total number of $\xi$'s |
| det of `Trs` | can be 0 when the system unobservable |

## 4.4 P4_iterative_calcu.m

For each $\xi$ in `init_sum`, the iteration (6) is applied for `knum` times. The command line
`ode15s(@i_directforx, [time_ini_x, stoptime], ini, options)`
solves the $x$-part in the positive time direction and
`ode15s(@i_directforp, [time_ini_p 0], zeros(dim,1), options)`
solves the $p$-part in the positive time direction. The negative time direction for enhancing the size of the stable manifold is computed by
`ode45(@i_canonical_eq, [Time_s Tneg], Tneg_ini_xpT, options),`
the initial value of which is a value of the stable manifold computed by the positive time computation. The size of the enhancement is controlled by `x_lim` in `systemData.m`.

`ini`: An element $\xi$ in `init_sum`

`stoptime`: Time interval for positive directional integration. Given in a form `[0, stoptime]`.

`ini_sum`: Values $x_k(0)$, $p_k(0)$ for $k$ = `knum`. Used in `SP_data_conv.m` and `SP_graphplot1.m` after converted into `ini_sumT`.

`txpN_i` (N = 0,...,`knum` -1, `knum`; i = 1,...,m): Computed data $t$, $x_k(t, \xi)$ and $p_k(t, \xi)$ for $k = M$, $m$ is the number of $\xi$'s.

$$\texttt{txpM_i} = [t, x_k, p_k], (\text{number of time sample} \times 2\texttt{dim} + 1) \text{ matrix}$$

`txpNsum` ($N = 0, ...,$ `knum`): Consists of `txpN_i` stacked from $i = 1, \ldots, m$

`txp`: The results of the computation with the following structure.

$$\texttt{txp} = [t, x_k, p_k]$$

## 4.5 P5_solution_check.m

From computed $x_k(t,\xi)$ and $p_k(t,\xi)$, the function $p(x)$ and the optimal feedback control are obtained. Finally, a closed loop simulation is performed with the controller.

**SP_data_conv.m**

The stable manifold algorithm is applied in the space where Hamiltonian matrix is block diagonalized. All the data needs to be converted to the original space. Note that variables with "T" such as txpNsumT at the end of their names are transformed ones into the original space.

txpNsumT ($N = 0, ...,$knum): Defined as (Diagonalized $\cdot$ (txpNsum)$^T$)$^T$. It is a tall matrix of the form $[t, x_k, p_k]$. This variable, especially when N = knum, is very important for data analysis.

ini_sumT: Defined as (Diagonalized $\cdot$ (ini_sum)$^T$)$^T$. Used only for graph plotting.

**SP_ham_calcu.m**

From txpNsumT, values of Hamiltonian are computed and ploted in figures.

HAMK0: Hamiltonian values for $k = 0$(linear solution).

HAMK: Hamiltonian values for $k =$knum. Important!

**SP_graphplot1.m**

This program shows, from txpNsumT($N = 1, ..,$knum), several figures to confirm the data distribution in $x$-space and behaviors of $p$ which are related to the optimal feedback controller.

**SP_data_cut.m**

Computed data around the origin is too dense and it just corresponds to the Riccati solution. Therefore, this program removes the computed data for $x_k(t,\xi)$ and $p_k(t,\xi)$, $t \gg 0$ in txpNsumT($N =$ knum).

txpNsumTC ($N =$ knum): The new data after the removal from txpNsumT.

**SP_multiPoly.m**

This program computes $p(x)$ as a function of $x$ from txpNsumTC by the multi-variable polynomial approximation.

Nx: This variable specifies the order of each variable. For $N$ dimensional polynomial, it has the following form.

$$\text{Nx} = [\text{ order of } x_1, \text{ order of } x_2, ..., \text{ order of } x_N \text{ }]$$

xc: Specifies the shape of polynomial function. For example, for 2-variable, 2nd order polynomials

$$p_1(x_1, x_2) = a_0 + a_1x_1 + a_2x_1^2 + a_3x_2 + a_4x_1x_2 + a_5x_2^2$$
$$p_2(x_1, x_2) = b_0 + b_1x_1 + b_2x_1^2 + b_3x_2 + b_4x_1x_2 + b_5x_2^2$$

it takes the following form.

$$\texttt{xc} = \begin{bmatrix} 0 & 1 & 2 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 2 \end{bmatrix}$$

`CN`: $N = 1, ..,$dimension of the range space. These are vectors that consist of coefficients of each terms. For the above example, they take

$$\texttt{C1} = [a_0, a_1, a_2, a_3, a_4, a_5, ]^T$$
$$\texttt{C2} = [b_0, b_1, b_2, b_3, b_4, b_5, ]^T .$$

**SP_griddata.m**

This computes $p(x)$ from `txpNsumTC` by the $n$-dimensional interpolation. Not recommended for systems

`Xrange`: : The range of $x_k$ for which interpolation is applied.

`Xspan`: Division spacing gap for `Xrange`

`p0Ng`: $N = 1, ..,$system dimension. Created data for $k = 0$ (linear solution).

`pNg`: $N = 1, ..,$system dimension. Created data for $k =$`knum`.

**SP_response_calcu.m**

This program performs simulations of the closed loop system with the obtained optimal control from the computed $p(x)$ and shows time responses and other data.

`a`: The initial condition for the simulation. Defined in `systemData.m`.

`t`: Time sample data of the response for $k =$`knum`.

`f`: Corresponding state data ($x$) for $k =$`knum` of the form $[x_1, x_2, ..., x_N]$, ($N =$system dimension). A tall matrix with $N$ columns.

`px`: Also, corresponding costate ($p$) data for $k =$`knum`. A tall matrix $[p_1, p_2, ..., p_N]$ with $N =$ columns.

`opt_u`: Corresponding optimal input history for $k =$`knum`.

`t0`: Time sample data for $k = 0$ (linear optimal control).

`f0`: Corresponding state data ($x$) for $k = 0$ (linear optimal control) with the same structure with `f`.

`opt_u0`: Time history of linear optimal control.

`SP_norm_calcu.m`

This program computes the norms of states and inputs at each time sample and show their time history.

`norm_x_Nth`: Norm of $x$ for $k =$`knum`

`norm_u_Nth`: Norm of the optimal input for $k =$`knum`

`norm_x_0th`: Norm of $x$ for $k = 0$ (linear optimal control)

`norm_u_0th`: Norm of the linear optimal input for $k = 0$.

# References


[1] S. C. Beeler, H. T. Tran, and H. T. Banks. Feedback control methodologies for nonlinear systems. *Journal of Optimization Theory and Applications*, 107(1):1–33, 2000.

[2] Takamasa Horibe and Noboru Sakamoto. Optimal swing up and stabilization control for inverted pendulum via stable manifold method. *IEEE Trans. on Control System Technology*, 26(2):708–715, 2017.

[3] Noboru Sakamoto. Case studies on the application of the stable manifold approach for nonlinear optimal control design. *Automatica*, 49(2):568–576, 2013.

[4] Noboru Sakamoto and Arjan J. van der Schaft. Analytical approximation methods for the stabilizing solution of the Hamilton-Jacobi equation. *IEEE Trans. Automat. Control*, 53(10):2335–2350, 2008.

[5] A. J. van der Schaft. On a state space approach to nonlinear $H_\infty$ control. *Syst. Control Lett.*, 16(1):1–18, 1991.